



FilmScoper

Descubre tu próxima película favorita

 DJANGO

5.2.6

 PYTHON

3.8+

 BOOTSTRAP

5.3.0

 TMDB

API V3

Una plataforma web moderna para descubrir, explorar y calificar películas

- ✳️ Características Principales ✳️
- 🔥 Sistema de Trailers Oficial con TMDB API | 🎯 Calificaciones Inteligentes | 💬 Comentarios Avanzados | 👤 Perfiles Personalizados

🚀 Estado del Proyecto: COMPLETO ☒

28 películas • 20 trailers oficiales • 6 categorías • Sistema completo de usuarios

💎 Ir a Instalación • 📄 Ver Características • 🎬 Trailers TMDB • 👤 Usuarios de Prueba

Índice

🎯 Información Esencial

- 👤 Usuarios de Prueba
- 💎 Descripción General
- 🚀 Instalación Rápida
- 📄 Últimas Actualizaciones

📊 Estado del Desarrollo

- 🎯 Alcance del Proyecto
- ✅ Funcionalidades Implementadas
- ⚠️ Funcionalidades en Desarrollo
- 📊 Progreso vs Actividad

🔧 Documentación Técnica

- 🔑 Tecnologías Utilizadas
- 📁 Estructura del Proyecto
- 🔍 Comandos Útiles
- 🔗 APIs REST Completas

👨‍💻 Para Desarrolladores

- 📞 Información del Desarrollador
- 📄 Notas de Implementación
- 🎮 Uso de la Aplicación

-  Ejecutar Pruebas
-  Tecnologías y Dependencias
-  Modelos de Datos
-  Despliegue en Producción

 APIs y Servicios Externos

-  Sistema TMDb
-  YouTube API
-  Endpoints de Películas
-  Endpoints de Reseñas
-  APIs de Estadísticas
-  Servicios Externos

 Información Adicional

-  Contribución
-  Licencia

 Navegación Interactiva

 **Tip:** Todos los títulos de este README son enlaces clicables. Usa **Ctrl+Click** para abrir en nueva pestaña.

 [Ir a Navegación Rápida](#) |  [Volver al Índice](#) |  [Ir al Inicio](#)

 Usuarios de Prueba Disponibles

Para Evaluación y Testing

El proyecto incluye usuarios pre-configurados para facilitar la evaluación:

Usuario	Email	Tipo	Contraseña	Descripción
admin	admin@filmscopers.com	Administrador	Adminx123.	Acceso completo al panel admin /admin/
User1	Email ejemplo@filmscopers.com	Usuario normal	Prueba123x.	Usuario estándar con datos de prueba

 Accesos Rápidos

- **Panel de Administración:** <http://127.0.0.1:8000/admin/>
- **Aplicación Principal:** <http://127.0.0.1:8000/>
- **Categorías:** Cada categoría tiene películas pre-cargadas
- **Listas Personales:** Los usuarios tienen favoritos y listas configuradas

 Datos de Prueba Incluidos

- ☒ **28 películas activas** distribuidas en 6 categorías (Acción, Comedia, Documentales, Romance, Terror, Infantil)
 - ☒ **20 trailers oficiales** integrados con TMDB API
 - ☒ **22 calificaciones oficiales** (IMDb) pre-cargadas
 - ☒ **10+ reseñas de usuarios** con calificaciones activas
 - ☒ **Sistema de comentarios avanzado** con límites por usuario y respuestas anidadas
 - ☒ **5 foros por categoría** auto-generados
 - ☒ **Usuarios con perfiles** completos y géneros favoritos
 - ☒ **Listas personalizadas** con películas agregadas
 - ☒ **Imágenes y contenido** listo para usar
-

Descripción

FilmScoper es una plataforma web moderna desarrollada con Django que permite a los usuarios descubrir, explorar y calificar películas. El proyecto integra funcionalidades avanzadas de gestión de usuarios, sistema de reseñas, categorización de películas y una interfaz de usuario elegante con Bootstrap 5. **Nueva característica:** Integración completa con TMDB API para trailers oficiales de películas.

Últimas Actualizaciones - Octubre 2025

Página de Inicio Mejorada con Rankings TMDB

- ☒ **Top 12 Películas:** Página principal muestra automáticamente las mejores películas por calificación TMDB
- ☒ **Posters Oficiales:** Sistema mejorado de visualización de imágenes con URLs TMDB
- ☒ **Fallbacks Elegantes:** Placeholders con gradientes y iconos SVG para películas sin poster
- ☒ **Efectos Interactivos:** Hover animations y transiciones suaves en las tarjetas
- ☒ **Design Responsivo:** Optimizado para todas las pantallas con Bootstrap Grid

Optimización y Limpieza del Proyecto

- ☒ **Eliminación de Redundancias:** Removidos archivos no utilizados (API REST, TMDB portadas obsoletas)
- ☒ **Estructura Limpia:** Proyecto optimizado sin componentes innecesarios
- ☒ **URLs Simplificadas:** Rutas limpias enfocadas en funcionalidad core
- ☒ **Rendimiento Mejorado:** Carga más rápida sin dependencias obsoletas

Sistema de Trailers Oficial con TMDB

- ☒ **TMDB API Integration:** Conexión completa con The Movie Database API v3
- ☒ **Trailers Oficiales:** 20 películas con trailers oficiales verificados por TMDB
- ☒ **YouTube Embedding:** Reproducción directa de trailers en modales Bootstrap
- ☒ **Gestión Automática:** Comando Django para actualización masiva de trailers
- ☒ **Fallback System:** Enlace directo a YouTube cuando el embed falla por restricciones
- ☒ **Caching Inteligente:** Sistema de caché para optimizar llamadas a la API

Importante: Restricciones de Reproducción de YouTube

Nota sobre limitaciones del sistema de trailers: Debido a las políticas de YouTube y restricciones impuestas por los estudios cinematográficos, algunos trailers oficiales no pueden reproducirse en formato embed (iframe) en sitios externos. Esto es una limitación de la plataforma YouTube, no del sistema FilmScoper.

Trailers que Sí funcionan correctamente:

- 🎬 **Hércules** - Reproducción completa disponible
- 🎬 **Mi Maestro Pulpo** - Funciona sin restricciones
- 🎬 **El Viaje del Emperador** - Reproducción exitosa
- 🎬 **La La Land** - Trailer disponible en embed
- 🎬 **Y varios más** - Muchos otros trailers funcionan perfectamente

Para trailers restringidos: El sistema proporciona automáticamente un **enlace directo a YouTube** donde los usuarios pueden ver el trailer completo sin restricciones. Esta es la mejor solución técnica disponible dado las limitaciones de la API de YouTube para contenido protegido por derechos de autor.

🔑 Nuevos Servicios y Comandos

- **TMDBService:** Clase completa para interactuar with TMDB API
 - Búsqueda de películas por título y año
 - Obtención de trailers oficiales
 - Gestión de imágenes y metadatos
 - Sistema de caché para optimización
- **Comando `actualizar_trailers_tmdb`:**
 - Actualización masiva de trailers desde TMDB
 - Opciones: `--todas`, `--limite N` para control granular
 - Progreso en tiempo real y manejo de errores
- **Modal de Trailers:** Diseño limpio con Bootstrap 5, responsivo y accesible

💡 Mejoras Técnicas Implementadas

- **Services Layer:** Arquitectura de servicios para APIs externas
- **Management Commands:** Comandos Django customizados para administración
- **Error Handling:** Manejo robusto de errores de API y restricciones de YouTube
- **Progressive Enhancement:** Funcionalidad que mejora la experiencia sin romper la base

🎯 Alcance y Estado del Proyecto

☑️ Funcionalidades Completamente Implementadas

📊 Base de Datos y Contenido

- ✅ **Catálogo de Películas:** 28 películas activas en 6 categorías (Acción, Comedia, Documentales, Romance, Terror, Infantil)
- ✅ **Sistema de Trailers:** 20 trailers oficiales integrados con TMDB API
- ✅ **Calificaciones Duales:** Sistema IMDb/Oficial + Promedio FilmScoper
- ✅ **Comandos de Poblado:** Scripts automatizados para llenar la base de datos

Gestión de Usuarios

- ☒ **Sistema de Usuarios:** Registro, login, logout con validaciones Django completas
- ☒ **Perfiles Personalizados:** Gestión de información personal, foto de perfil y géneros favoritos
- ☒ **Listas Personalizadas:** Sistema de favoritos y "Ver más tarde" completamente funcional
- ☒ **Protección de Rutas:** Decoradores `@login_required` para vistas sensibles

Interfaz y Experiencia de Usuario

- ☒ **Interfaz Responsiva:** Compatible con dispositivos móviles y desktop con Bootstrap 5
- ☒ **Navegación por Categorías:** Filtrado y exploración intuitiva
- ☒ **Paginación Optimizada:** 12 películas por página con navegación fluida
- ☒ **Modales de Trailers:** Reproducción integrada con fallback a YouTube directo

★ Sistema de Reseñas y Comentarios

- ☒ **Calificaciones con Estrellas:** Interfaz AJAX para calificar películas (1-5 estrellas)
- ☒ **Sistema Avanzado de Comentarios:**
 - Límites inteligentes: 5 comentarios padre + 10 respuestas por usuario por película
 - Respuestas anidadas con threading completo
 - Rotación automática de comentarios con controles manuales
 - Validación completa de formularios con corrección de errores
 - Interfaz compacta con diseño optimizado

Arquitectura y Seguridad

- ☒ **Validación de Formularios:** Django Forms con validación híbrida cliente/servidor
- ☒ **CSRF Protection:** Protección contra ataques de falsificación de solicitudes
- ☒ **Panel de Administración:** Django Admin configurado para gestión completa de contenido
- ☒ **Tests Automatizados:** Suite de pruebas para modelos y vistas principales
- ☒ **Services Architecture:** Capa de servicios para integraciones externas

APIs y Servicios Externos

- ☒ **TMDB API Integration:** Servicio completo para The Movie Database
- ☒ **YouTube Integration:** Embedding de trailers con manejo de restricciones
- ☒ **Caching System:** Sistema de caché para optimizar rendimiento de APIs

Funcionalidades Parcialmente Implementadas

- ☒ **Notificaciones:** Sistema básico con SweetAlert2 (solo para listas personales)
- ☐ **Sistema de Búsqueda:** Frontend implementado pero sin funcionalidad backend
- ☐ **Filtros Avanzados:** Interfaz creada pero sin lógica de filtrado

✗ Funcionalidades Pendientes (No Implementadas)

- ☐ **Interfaz de Foros:** Templates y vistas para navegación de foros (backend completo)
- ☐ **Sistema de Recomendaciones:** Algoritmos de recomendación personalizados
- ☐ **Notificaciones Push:** Sistema de notificaciones en tiempo real

- ☐ **API REST:** Endpoints para aplicaciones móviles o terceros
- ☐ **Sistema de Moderación Avanzado:** Más allá del Django Admin básico

Instalación y Configuración

Prerrequisitos

- **Python 3.8 o superior:** Lenguaje de programación principal
- **pip:** Gestor de paquetes de Python (incluido con Python)
- **Git:** Para clonar el repositorio (opcional si descargas ZIP)

Instalación Completa - Paso a Paso

1. Clonar el Repositorio

```
git clone https://github.com/NathanielMuller/FilmScoperProyect.git
cd FilmScoperProyect
```

2. Crear Entorno Virtual (Recomendado)

```
# Windows
python -m venv venv
venv\Scripts\activate

# Linux/Mac
python -m venv venv
source venv/bin/activate
```

3. Instalar Dependencias de Python

Librerías Principales

```
# Framework principal
pip install Django==5.2.6

# Formularios y UI
pip install django-crispy-forms==2.3
pip install crispy-bootstrap5==2024.2

# APIs Externas y HTTP
pip install requests==2.32.3

# Base de datos y cache
pip install pillow==10.4.0 # Para manejo de imágenes de perfil

# Desarrollo y testing (opcional)
pip install django-debug-toolbar # Para debugging en desarrollo
```

O Instalar Todas de una Vez

```
# Si tienes un archivo requirements.txt (crear si no existe)
pip install -r requirements.txt
```

4. Configuración de TMDB API (Nuevo)

Obtener API Key de TMDB

1. **Registrarse en TMDB:** <https://www.themoviedb.org/signup>
2. **Solicitar API Key:**
 - Ir a Settings → API
 - Solicitar una API Key (gratuita)
 - Tipo: Developer
3. **Configurar en Django:**
 - Abrir `film_scoper/settings.py`
 - Añadir: `TMDB_API_KEY = 'tu_api_key_aqui'`
 - O usar variables de entorno (recomendado para producción)

Configuración de Variables de Entorno (Opcional)

```
# Crear archivo .env en la raíz del proyecto
TMDB_API_KEY=tu_api_key_de_tmdb_aqui
SECRET_KEY=tu_secret_key_de_django
DEBUG=True
### 5. Configuración de Base de Datos

```bash
Ejecutar migraciones para crear la estructura de BD
python manage.py migrate

Crear superusuario para el panel admin
python manage.py createsuperuser
```

## 6. Poblar Base de Datos con Datos de Prueba

### Comandos de Poblado Automático

```
Poblar películas iniciales (28 películas en 6 categorías)
python manage.py poblar_peliculas

Agregar calificaciones oficiales IMDb
python manage.py poblar_calificaciones
```

```
Crear foros por categorías
python manage.py crear_foros

📺 NUEVO: Actualizar trailers desde TMDB API
python manage.py actualizartrailers_tmdb --todas
```

🔧 Comandos Específicos de Trailers

```
Actualizar solo películas sin trailer
python manage.py actualizartrailers_tmdb

Actualizar todas las películas (recomendado)
python manage.py actualizartrailers_tmdb --todas

Limitar a N películas para pruebas
python manage.py actualizartrailers_tmdb --limite 5
```

7. Ejecutar el Servidor

```
python manage.py runserver
```

☑ **La aplicación estará disponible en:** <http://127.0.0.1:8000/>

🔗 Tecnologías y Librerías Utilizadas

🔧 Backend - Python/Django

Librería	Versión	Propósito
Django	5.2.6	Framework web principal
django-crispy-forms	2.3	Formularios elegantes y responsivos
crispy-bootstrap5	2024.2	Integración Bootstrap 5 con crispy-forms
requests	2.32.3	<b>NUEVO:</b> Llamadas HTTP a APIs externas (TMDB)
Pillow	10.4.0	Procesamiento de imágenes para ImageField

🌐 Frontend - HTML/CSS/JavaScript

Tecnología	Versión	Propósito
Bootstrap	5.3.0	Framework CSS responsivo
SweetAlert2	11.x	Notificaciones elegantes



Tecnología	Versión	Propósito
Font Awesome	6.x	Iconografía
Google Fonts	-	Tipografías (Lexend, Lato)
JavaScript Vanilla	ES6+	Interactividad del cliente

 APIs Externas (Nuevas)

Servicio	Propósito	Estado
TMDB API v3	Trailers oficiales de películas	<input checked="" type="checkbox"/> Implementado
YouTube Embed API	Reproducción de trailers	<input checked="" type="checkbox"/> Implementado

 Base de Datos

- **SQLite3:** Base de datos por defecto (incluida en Django)
- **Modelos Django ORM:** Para abstracción de base de datos

 Estructura del Proyecto

```
FilmScoperProject/
├── core/ # Aplicación principal
│ ├── management/commands/ # ★ Comandos Django personalizados
│ │ ├── poblar_peliculas.py # Poblar películas iniciales
│ │ ├── poblar_calificaciones.py # Calificaciones oficiales
│ │ ├── crear_foros.py # Crear foros por categoría
│ │ └── actualizartrailers_tmdb.py # 🎬 Trailers TMDB automáticos
│ ├── migrations/ # Migraciones de base de datos
│ ├── static/core/ # Archivos estáticos
│ │ ├── css/ # Estilos personalizados
│ │ ├── js/ # JavaScript del frontend
│ │ └── img/ # Imágenes de películas
│ ├── templates/core/ # Templates HTML optimizados
│ │ ├── base.html # Template base con Bootstrap 5
│ │ ├── index.html # 🏠 Página principal con top películas
│ │ ├── categoria.html # Listados por categoría
│ │ ├── pelicula.html # Detalles con trailers TMDB
│ │ └── ... # Otros templates del sistema
│ ├── templatetags/ # Tags personalizados de Django
│ └── models.py # 📊 Modelos optimizados (get_poster_url mejorado)
├── views.py # 🔄 Vistas mejoradas (index con top ranking)
├── urls.py # 🔗 URLs simplificadas y limpias
├── forms.py # Formularios Django
├── admin.py # Configuración del panel admin
├── services.py # 🎬 Servicios TMDB API
├── signals.py # Señales Django para automatización
└── film_scoper/ # Configuración del proyecto Django
```

```

| | settings.py # ⚙️ Configuraciones + TMDB API Key
| | urls.py # URLs principales (simplificadas)
| | wsgi.py # Configuración WSGI
| db.sqlite3 # 📄 Base de datos SQLite (poblada)
| manage.py # Script de gestión de Django
| README.md # 📖 Documentación completa

```

## ✂ Archivos Eliminados en Optimización:

- ✗ `core/api_urls.py`, `core/api_views.py`, `core/serializers.py` - API REST no implementada
- ✗ `core/tmdb_views.py`, `core/templates/admin/buscar_portadas_tmdb.html` - Sistema TMDB obsoleto
- ✗ `instrucciones/` - Documentación de desarrollo ya no necesaria

## 🎯 Progreso de Desarrollo - Actividad Evaluativa

☑ Requerimientos Cumplidos Completamente

### 📺 R01 - Gestión de Películas

- ☑ Modelos de datos completos (Película, Género, Categoría)
- ☑ CRUD completo a través de Django Admin
- ☑ 28 películas de prueba distribuidas en 6 categorías
- ☑ **BONUS:** Integración con TMDB API para trailers oficiales
- ☑ **NUEVO:** Página de inicio con top 12 películas por ranking TMDB

### 👤 R02 - Sistema de Usuarios

- ☑ Registro, login, logout con validaciones Django
- ☑ Perfiles extendidos con foto y géneros favoritos
- ☑ Protección de rutas con `@login_required`
- ☑ Sistema de permisos y roles

### ★ R03 - Sistema de Calificaciones

- ☑ Calificaciones de 1-5 estrellas con AJAX
- ☑ Sistema dual: Calificación oficial (IMDb) + Promedio FilmScoper
- ☑ 22 calificaciones oficiales pre-cargadas
- ☑ Interfaz de usuario optimizada y responsiva

### 💬 R04 - Sistema de Comentarios

- ☑ Comentarios con respuestas anidadas (threading)
- ☑ Límites por usuario: 5 comentarios + 10 respuestas por película
- ☑ Rotación automática con controles manuales
- ☑ Validación completa de formularios
- ☑ Interfaz compacta y moderna

## R05 - Categorización

- ☒ 6 categorías: Acción, Comedia, Documentales, Romance, Terror, Infantil
- ☒ Navegación por categorías con paginación
- ☒ Filtrado automático y manual
- ☒ Distribución balanceada de contenido

## R06 - Interfaz Responsiva

- ☒ Bootstrap 5 completamente integrado
- ☒ Diseño mobile-first y responsive
- ☒ Componentes optimizados para todas las pantallas
- ☒ Tipografías modernas (Google Fonts: Lexend, Lato)

## R07 - Panel de Administración

- ☒ Django Admin configurado y personalizado
- ☒ Gestión completa de películas, usuarios y contenido
- ☒ Usuarios de prueba para evaluación
- ☒ Comandos Django para automatización

## Mejoras y Características Adicionales

### Nueva Integración TMDB API

- ☒ Conexión completa con The Movie Database API v3
- ☒ 20 trailers oficiales actualizados automáticamente
- ☒ Comando Django personalizado para gestión masiva
- ☒ Sistema de caché para optimización de rendimiento
- ☒ Manejo de errores y restricciones de YouTube

### Listas Personalizadas

- ☒ Sistema de favoritos completamente funcional
- ☒ Lista "Ver más tarde" con gestión AJAX
- ☒ Notificaciones con SweetAlert2
- ☒ Persistencia en base de datos

### Arquitectura Avanzada

- ☒ Services Layer para APIs externas (**TMDBService**)
- ☒ Management Commands personalizados
- ☒ Sistema de señales Django para automatización
- ☒ Template tags personalizados para funcionalidades específicas

### APIs REST Implementadas

- ☒ **Django REST Framework** completamente configurado
- ☒ **20+ Endpoints REST** para acceso programático a datos

- ☒ **Autenticación y Permisos** granulares por endpoint
- ☒ **Paginación Automática** y filtros avanzados
- ☒ **Documentación API** integrada con navegador web

## APIs de Servicios Externos

- ☒ **TMDB Integration:** Búsqueda de portadas y metadatos de películas
- ☒ **YouTube API:** Búsqueda y gestión de trailers oficiales
- ☒ **Manejo de Errores:** Sistema robusto para APIs externas
- ☒ **Caché Inteligente:** Optimización de llamadas a servicios externos

## Funcionalidades en Desarrollo

- 🔍 **Sistema de Búsqueda:** Frontend implementado, backend pendiente
- 🔧 **Filtros Avanzados:** UI creada, lógica de filtrado pendiente

## Métricas del Proyecto Final

- 📄 **Líneas de Código:** ~3,200 líneas (optimizado, -300 por limpieza)
- 🗃️ **Modelos:** 8 modelos de datos principales
- 📄 **Templates:** 10+ templates HTML optimizados
- ⚙️ **Comandos:** 4 comandos Django personalizados
- 🧪 **Tests:** Suite de pruebas automatizadas
- 🎬 **APIs:** Integración con 2 APIs externas (TMDB + YouTube)
- ⌚ **Funcionalidades:** Página inicio con ranking automático TMDB
- ✂️ **Optimización:** Proyecto limpio sin archivos redundantes

## Comandos Útiles para Desarrollo

### Gestión de Datos

```
Poblar base de datos completa
python manage.py poblar_peliculas
python manage.py poblar_calificaciones
python manage.py crear_foros

🎬 Gestión de trailers TMDB
python manage.py actualizar_trailers_tmdb --todas --limite 10
```

### Desarrollo y Testing

```
Ejecutar tests
python manage.py test

Crear migraciones cuando cambies modelos
python manage.py makemigrations
python manage.py migrate
```

```
Recolectar archivos estáticos (producción)
python manage.py collectstatic
```

## 🔑 Depuración

```
Shell interactivo de Django
python manage.py shell

Verificar configuración
python manage.py check

Ver estructura de BD
python manage.py dbshell
```

## ☎ Información del Desarrollador

- **Estudiante:** Nathaniel Muller
- **Institución:** DUOC UC
- **Programa:** Programación Web
- **Semana:** 8
- **Proyecto:** FilmScoper - Plataforma de Descubrimiento de Películas

## 📄 Notas de Implementación

### 🎬 TMDB API Configuration

Para utilizar los trailers, necesitas:

1. API Key de TMDB (gratuita en <https://www.themoviedb.org/>)
2. Configurar en `settings.py`: `TMDB_API_KEY = 'tu_key'`
3. Ejecutar: `python manage.py actualizar_trailers_tmdb --todas`

## 🚀 Para Producción

- Cambiar `DEBUG = False` en `settings.py`
- Configurar base de datos PostgreSQL/MySQL
- Usar variables de entorno para secrets
- Configurar servidor web (Apache/Nginx + Gunicorn)
- Configurar dominio y SSL

---

## 🍷 Documentación Completa de APIs REST

### 📖 Información General de la API

FilmScoper incluye una **API REST completa** desarrollada con **Django REST Framework** que permite acceso programático a todas las funcionalidades del sistema. La API está diseñada para ser **RESTful**, **escalable** y **fácil**

de integrar con aplicaciones externas.

🌐 **Base URL:** <http://127.0.0.1:8000/api/>

## 🏗️ Arquitectura de la API

### 🔧 Tecnologías Utilizadas

- **Django REST Framework:** Framework principal para APIs REST
- **Session Authentication:** Para usuarios web autenticados
- **Token Authentication:** Para acceso programático (futuro)
- **Paginación Automática:** 12 elementos por página por defecto
- **Filtros y Búsqueda:** Integrados en todos los endpoints de listado

### 🔑 Autenticación y Permisos

- **Lectura Pública:** La mayoría de endpoints permiten lectura sin autenticación
- **Escritura Autenticada:** Crear/modificar requiere usuario autenticado
- **Permisos Granulares:** Diferentes niveles según el tipo de operación
- **Validación Automática:** Solo autores pueden modificar su contenido

## 🎬 Endpoints de Películas

### 📁 Lista y Gestión de Películas

GET	/api/peliculas/	# Lista todas las películas activas
POST	/api/peliculas/	# Crear nueva película (solo staff)
GET	/api/peliculas/{slug}/	# Detalles de película específica
PUT	/api/peliculas/{slug}/	# Actualizar película (solo staff)
DELETE	/api/peliculas/{slug}/	# Eliminar película (soft delete, solo staff)

### 🔍 Filtros Disponibles para /api/peliculas/:

- `?search=termino` - Búsqueda en título, director, reparto, sinopsis
- `?categorias__slug=accion` - Filtrar por categoría específica
- `?año=2020` - Filtrar por año de producción
- `?director=nombre` - Filtrar por director
- `?ordering=-calificacion_promedio` - Ordenar por calificación descendente
- `?page=2` - Navegación por páginas

### 📄 Películas por Categoría

GET	/api/peliculas/categoria/{categoria_slug}/	# Películas de categoría específica
-----	--------------------------------------------	-------------------------------------

 **Búsqueda Avanzada**

```
GET /api/buscar/?q=termino&categoria=accion&año=2020&minimo_rating=4
```

 **Endpoints de Reseñas**

 **Gestión de Reseñas**

```
GET /api/reseñas/ # Lista todas las reseñas activas
POST /api/reseñas/ # Crear nueva reseña (autenticado)
GET /api/reseñas/{id}/ # Detalles de reseña específica
PUT /api/reseñas/{id}/ # Actualizar reseña (solo autor)
DELETE /api/reseñas/{id}/ # Eliminar reseña (solo autor)
```

 **Reseñas por Usuario y Película**

```
GET /api/reseñas/pelicula/{pelicula_slug}/ # Reseñas de película específica
GET /api/reseñas/usuario/{username}/ # Reseñas de usuario específico
```

 **Características Especiales:**

- **Validación Anti-Duplicados:** Un usuario solo puede reseñar una película una vez
- **Actualización Automática:** Las calificaciones de películas se actualizan automáticamente
- **Soft Delete:** Las reseñas eliminadas mantienen integridad referencial

 **Endpoints de Categorías**

```
GET /api/categorias/ # Lista todas las categorías disponibles
```

 **Endpoints de Estadísticas**

 **Estadísticas Generales**

```
GET /api/estadisticas/peliculas/ # Estadísticas del sitio completo
```

 **Datos Incluidos:**

- Total de películas activas
- Películas por categoría
- Promedio general de calificaciones

- Película mejor calificada
- Películas agregadas recientemente

## Estadísticas de Usuario

```
GET /api/estadisticas/usuario/ # Estadísticas del usuario autenticado
```

### Datos Personales:

- Total de reseñas realizadas
- Promedio de calificaciones otorgadas
- Cantidad de películas favoritas
- Número de comentarios realizados

## APIs de Servicios Externos

### TMDb (The Movie Database)

#### Búsqueda de Portadas

```
GET /api/tmdb/buscar-portadas/?titulo=inception&año=2010
```

#### Asignar Portada a Película

```
POST /api/tmdb/asignar-portada/
Content-Type: application/json

{
 "pelicula_id": 123,
 "tmdb_poster_path": "/path/to/poster.jpg",
 "tmdb_id": 456
}
```

### Películas Populares TMDb

```
GET /api/tmdb/populares/?page=1 # Películas populares de TMDb
```

### YouTube API para Trailers

#### Búsqueda de Trailers

```
GET /api/youtube/buscar-trailers/?movie_title=inception&year=2010
```



## Detalles de Trailer Específico

```
GET /api/youtube/trailer/?youtube_id=YoHD9XEInc0
```

## Trailer de Película

```
GET /api/peliculas/{pelicula_id}/trailer/ # Info del trailer de una película
específica
```

## Uso Práctico de las APIs

### Ejemplo de Integración

```
// Obtener películas de acción con alta calificación
fetch('/api/peliculas/?categorias__slug=accion&ordering=-calificacion_promedio')
 .then(response => response.json())
 .then(data => {
 console.log('Películas de acción top:', data.results);
 });

// Crear una nueva reseña
fetch('/api/reseñas/', {
 method: 'POST',
 headers: {
 'Content-Type': 'application/json',
 'X-CSRFToken': getCookie('csrftoken') // Para usuarios web
 },
 body: JSON.stringify({
 pelicula: 'pelicula-slug',
 calificacion: 5,
 comentario: 'Excelente película!'
 })
});
```

## Casos de Uso Comunes

- **Apps Móviles:** Consumir datos para aplicaciones nativas
- **Dashboards:** Crear paneles de control con estadísticas
- **Integraciones:** Conectar con otros sistemas de películas
- **Bots:** Automatizar gestión de contenido y moderación
- **Análisis:** Extraer datos para análisis y reportes

## Seguridad y Validaciones

## Medidas de Seguridad Implementadas

- **CSRF Protection:** Protección contra ataques Cross-Site Request Forgery
- **Permisos por Método:** GET público, POST/PUT/DELETE autenticados
- **Validación de Propiedad:** Solo autores pueden modificar su contenido
- **Rate Limiting:** Próxima implementación para prevenir abuso
- **Sanitización:** Validación y limpieza automática de datos de entrada

## Limitaciones Actuales

- **Autenticación Simple:** Solo Session Auth (Token Auth planeado)
- **Rate Limiting:** No implementado aún
- **Versionado:** API v1 implícita (versionado formal futuro)

## APIs Externas Integradas

### TMDb (The Movie Database) API v3

**Propósito:** Obtener metadatos oficiales de películas

-  **Funcionalidad:** Búsqueda de películas por título y año
-  **Portadas:** URLs oficiales de posters en múltiples resoluciones
-  **Calificaciones:** Ratings oficiales de TMDb para el sistema
-  **Uso:** Comando `actualizar_trailers_tmdb` para sincronización masiva
-  **Cache:** Sistema de caché para optimizar llamadas repetidas

### YouTube Data API v3

**Propósito:** Gestión de trailers oficiales de películas

-  **Búsqueda:** Encontrar trailers oficiales por título de película
-  **Metadatos:** Información detallada de videos (duración, calidad, etc.)
-  **Embedding:** URLs optimizadas para reproducción en iframe
-  **Restricciones:** Manejo automático de limitaciones de reproducción
-  **Fallbacks:** Enlaces directos cuando el embed está restringido

### Flujo de Trabajo Completo YouTube:

1. **Búsqueda Automática:** El sistema busca trailers por título + año
2. **Validación de Calidad:** Filtra por palabras clave oficiales ("official", "trailer")
3. **Almacenamiento:** Guarda YouTube ID en base de datos
4. **Reproducción Inteligente:** Intenta embed, fallback a enlace directo
5. **Actualizaciones:** Comando para actualizar trailers masivamente

## Futuras Mejoras de la API

### Funcionalidades Planeadas

- **Token Authentication:** Autenticación por tokens para apps externas
- **Rate Limiting:** Limitación de requests por usuario/IP

- **API Versioning:** Versionado formal con `/api/v2/`
- **WebSocket Support:** Actualizaciones en tiempo real
- **GraphQL:** Endpoint GraphQL para queries flexibles
- **Swagger/OpenAPI:** Documentación interactiva automática

---

## FilmScoper - Descubre tu próxima película favorita ★

### 4. Configurar Base de Datos

```
python manage.py migrate
```

### 5. Cargar Datos de Ejemplo

```
python manage.py poblar_peliculas
python manage.py agregar_peliculas
python manage.py poblar_calificaciones
python manage.py crear_foros
```

### 6. Crear Superusuario (Opcional)

```
python manage.py createsuperuser
```

### 7. Ejecutar Servidor de Desarrollo

```
python manage.py runserver
```

La aplicación estará disponible en: <http://127.0.0.1:8000/>

## Estructura del Proyecto

```
FilmScoperProject/
├── film_scoper/ # Configuración principal del proyecto
│ ├── settings.py # Configuraciones de Django
│ ├── urls.py # URLs principales
│ └── wsgi.py # Configuración WSGI
├── core/ # Aplicación principal
│ ├── models.py # Modelos de datos
│ ├── views.py # Lógica de vistas
│ ├── forms.py # Formularios Django
│ ├── urls.py # URLs de la aplicación
│ └── admin.py # Configuración del admin
```

```
├── tests.py # Pruebas automatizadas
├── management/ # Comandos personalizados
│ └── commands/
│ ├── poblar_peliculas.py
│ └── agregar_peliculas.py
├── static/core/ # Archivos estáticos
│ ├── css/
│ ├── js/
│ └── img/
├── templates/core/ # Templates Django
│ ├── base.html
│ ├── index.html
│ ├── categoria.html
│ └── ...
├── db.sqlite3 # Base de datos SQLite
├── manage.py # Utilidad de Django
└── README.md # Este archivo
```

## Uso de la Aplicación

### Para Usuarios (Funcionalidades Disponibles)

1. **Registro:** Crear cuenta con email, fecha de nacimiento y géneros favoritos
2. **Exploración:** Navegar por categorías de películas con calificaciones duales
3. **Calificación:** Puntuar películas del 1-5 estrellas con interfaz AJAX integrada
4. **Comentarios Avanzados:**
  - Escribir hasta 5 comentarios principales por película
  - Responder hasta 10 veces por película con comentarios anidados
  - Sistema de rotación automática de comentarios con controles manuales
  - Formularios con validación completa y mensajes de confirmación
5. **Perfil:** Personalizar información personal y foto de perfil
6. **Listas Personalizadas:** Agregar/quitar películas de favoritos y "ver más tarde" con notificaciones
7. **Visualización Mejorada:** Ver detalles completos con ratings oficiales vs. comunidad, botones compactos y diseño responsive

### Para Administradores

1. **Panel Admin:** Acceder a `/admin/` con credenciales de superusuario
2. **Gestión de Contenido:** Agregar/editar películas y categorías
3. **Gestión de Usuarios:** Administrar cuentas y perfiles
4. **Comandos:** Usar comandos personalizados para poblar datos

### Limitaciones Actuales

- **Sin Búsqueda Funcional:** El buscador no procesa consultas
- **Sin Recomendaciones:** No hay sugerencias personalizadas a cada usuario

## Ejecutar Pruebas

```
Ejecutar todas las pruebas
python manage.py test

Ejecutar pruebas con verbosidad
python manage.py test -v 2

Ejecutar pruebas específicas
python manage.py test core.tests.PeliculaModelTest
```

## Tecnologías y Dependencias

### Backend - Python y Django

- **Django 5.2.6:** Framework web de Python para el desarrollo principal
- **SQLite:** Base de datos incorporada (incluida con Python)
- **django-crispy-forms:** Formularios elegantes y responsivos
- **crispy-bootstrap5:** Integración de crispy-forms con Bootstrap 5
- **Pillow:** Biblioteca de procesamiento de imágenes para ImageField

### Frontend

- **Bootstrap 5.3.7:** Framework CSS responsivo con diseño compacto y botones optimizados
- **SweetAlert2:** Notificaciones elegantes para confirmaciones de acciones
- **JavaScript ES6:**
  - Funcionalidades interactivas avanzadas
  - Sistema de rotación de comentarios con auto-play
  - Controles dinámicos para formularios de respuesta
  - Validación client-side integrada
- **CSS3:** Estilos personalizados con diseño responsive

### Funcionalidades Django Utilizadas

- **Django ORM:** Para manejo de base de datos y modelos
- **Django Forms:** Formularios con validación híbrida
- **Django Auth:** Sistema de autenticación y usuarios
- **Django Admin:** Panel de administración
- **Django Templates:** Sistema de plantillas
- **Django Static Files:** Manejo de archivos estáticos
- **Django Test Framework:** Pruebas automatizadas
- **Django Management Commands:** Comandos personalizados para poblar datos

### Herramientas de Desarrollo

- **Django Debug Toolbar:** Para debugging en desarrollo (opcional)
- **Git:** Control de versiones
- **VS Code:** Editor recomendado con extensiones de Python y Django

## Modelos de Datos

## Principales Entidades (Implementadas)

- **Usuario:** Sistema completo de autenticación Django
- **PerfilUsuario:** Información extendida con foto y géneros favoritos
- **Película:** Catálogo completo con posters, banners e información detallada
- **Categoría:** Sistema de clasificación por géneros
- **Listas Personales:** Favoritos y "Ver más tarde" (ManyToMany)

## Entidades Implementadas Completamente

- **Reseña:** Sistema completo de calificaciones (1-5 estrellas) con interfaz AJAX integrada
- **ComentarioPelícula:** Sistema avanzado de comentarios múltiples con límites por usuario (5+10), respuestas anidadas, rotación automática y validación completa
- **ForoCategoría/TemaDeForo/RespuestaForo:** Estructura completa de foros (sin interfaz)

## Relaciones Implementadas

- Usuario → PerfilUsuario (1:1) ☒
- Usuario → Listas Personales (M:M) ☒
- Película → Categoría (M:M) ☒
- Usuario → Reseña (1:N) ☒ *Interfaz completa con AJAX*
- Película → Reseña (1:N) ☒ *Promedio automático*
- Usuario → ComentarioPelícula (1:N) ☒ *Sistema avanzado con límites (5 padre + 10 respuestas)*
- Película → ComentarioPelícula (1:N) ☒ *Threading completo con rotación y validación*
- Categoría → ForoCategoría (1:1) ☒ *Backend completo*

## Relaciones Pendientes

- Usuario → Seguimiento (No planificado)
- Usuario → Notificaciones (No implementado)
- Película → Recomendaciones (No implementado)

## Despliegue en Producción

### Configuraciones Recomendadas

1. **Variables de Entorno:** Usar archivos `.env` para secretos
2. **Base de Datos:** PostgreSQL para producción
3. **Archivos Estáticos:** Configurar servidor web (Nginx)
4. **SSL:** Implementar HTTPS
5. **Monitoreo:** Logs y métricas de rendimiento

### Checklist de Seguridad

- ☐ `DEBUG = False` en producción
- ☐ `SECRET_KEY` en variable de entorno
- ☐ `ALLOWED_HOSTS` configurado
- ☐ Configurar cabeceras de seguridad
- ☐ Backup automático de base de datos

## Contribución

1. Fork del repositorio
2. Crear rama para feature (`git checkout -b feature/nueva-funcionalidad`)
3. Commit de cambios (`git commit -am 'Agregar nueva funcionalidad'`)
4. Push a la rama (`git push origin feature/nueva-funcionalidad`)
5. Crear Pull Request

## Licencia

Este proyecto está bajo la Licencia MIT. Ver el archivo [LICENSE](#) para más detalles.

## Contacto

- **Desarrollador:** Nathaniel Muller
- **GitHub:** [@NathanielMuller](#)
- **Proyecto:** [FilmScoperProject](#)

## Roadmap y Desarrollo Futuro

### Próximas Implementaciones (Prioridad Alta)

- ☐ **Interfaz de Foros:** Templates para navegación, creación de temas y moderación
- ☐ **Búsqueda Funcional:** Backend para búsqueda por título, género, año con filtros avanzados
- ☐ **Sistema de Moderación:** Reportes de comentarios, administración de contenido y moderación automática
- ☒ ~~Sistema de Comentarios Avanzado:~~ Límites, rotación y threading ☒ **COMPLETADO**

### Funcionalidades Planificadas (Prioridad Media)

- ☐ **API REST:** Para posibles aplicaciones móviles
- ☐ **Sistema de Recomendaciones:** Sugerencias basadas en preferencias
- ☐ **Integración con APIs de Películas:** TMDb para datos actualizados
- ☐ **Filtros Avanzados:** Por año, director, duración, calificación
- ☐ **Sistema de Seguimiento:** Seguir a otros usuarios
- ☐ **Estadísticas de Usuario:** Dashboard personal con métricas

### Ideas Futuras (Prioridad Baja)

- ☐ **Chat en Tiempo Real:** Comunicación entre usuarios
- ☐ **Sistema de Logros:** Gamificación de la experiencia
- ☐ **Modo Oscuro/Claro:** Personalización de tema
- ☐ **Listas Colaborativas:** Listas compartidas entre usuarios
- ☐ **Sistema de Moderación Avanzado:** Reportes y moderación automática
- ☐ **Aplicación Móvil:** App nativa para iOS/Android

### Mejoras Técnicas Pendientes

- ☐ **Optimización de Base de Datos:** Índices y consultas optimizadas
- ☐ **Sistema de Cache:** Redis para mejor rendimiento

-  **Logs y Monitoreo:** Sistema de logging completo
  -  **Tests de Integración:** Cobertura de testing al 90%+
  -  **Documentación de API:** Swagger/OpenAPI documentation
  -  **CI/CD Pipeline:** Automatización de despliegue
- 

## Navegación Rápida

### Enlaces Principales

-  [Volver al Inicio](#)
-  [Ver Índice Completo](#)
-  [Instalación Rápida](#)
-  [Usuarios de Prueba](#)

### Secciones Técnicas

-  [Documentación APIs REST](#)
-  [Sistema TMDb Trailers](#)
-  [Estado del Proyecto](#)
-  [Tecnologías Utilizadas](#)

### Para Desarrolladores

-  [Estructura del Proyecto](#)
  -  [Comandos Útiles](#)
  -  [Notas de Implementación](#)
  -  [Info del Desarrollador](#)
- 

## FilmScoper - Proyecto en Desarrollo Activo

*Este es un proyecto académico en desarrollo. Algunas funcionalidades mostradas en el diseño están pendientes de implementación. Consulta la sección "Alcance y Estado del Proyecto" para conocer qué está completamente funcional.*

**¡Gracias por explorar FilmScoper! **